

TiMBL: Tilburg Memory-Based Learner

version 7.0

API Reference Guide

Ko van der Sloot

Centre for Language Studies
Radboud University Nijmegen

URL: [`https://languagemachines.github.io/timbl`](https://languagemachines.github.io/timbl)¹

July 3, 2026

¹This document is available from [`https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl\_7.0\_API.pdf`](https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl_7.0_API.pdf).

Contents

1	Changes	3
1.1	From version 6.3 to 7.0	3
1.2	From version 6.2 to 6.3	4
1.3	From version 6.1 to 6.2	4
1.4	From version 5.1 to 6.1	4
1.5	From version 5.0 to 5.1	4
2	Quick-start	5
2.1	Setting up an experiment	5
2.2	Running an experiment	6
2.2.1	Training	6
2.2.2	Testing	6
2.2.3	Special cases of <code>Learn()</code> and <code>Test()</code>	6
2.3	More about settings	7
2.4	Storing and retrieving intermediate results	7
3	Classify functions	9
3.1	Classify functions: Elementary	9
3.2	Classify functions: Advanced	10
3.3	Classify functions: <code>neighborSets</code>	11
3.3.1	How to get results from a <code>neighborSet</code>	12
3.3.2	Useful operations on <code>neighborSet</code> objects	12
4	Advanced Functions	14
4.1	Modifying the <code>InstanceBase</code>	14

4.2	Pruning an InstanceBase to an IGTre	14
4.3	Getting more information out of Timbl	15
5	Server mode	16
6	Annotated example programs	17
6.0.1	example 1, api_test1.cxx	17
6.0.2	example 2, api_test2.cxx	20
6.0.3	example 3, api_test3.cxx	23
6.0.4	example 4, api_test4.cxx	25
6.0.5	example 5, api_test5.cxx	28
6.0.6	example 6, api_test6.cxx	31

Preface

This is a brief description of the `TimblAPI` class, the application programming interface to the `Timbl`¹ software package, and its main functions. For an introduction into `Timbl`, consult the `Timbl Reference Guide` [1]. Although most of the API can be traced in the `TimblAPI.h` file, the reverse is not true; some functions in `TimblAPI.h` are still “work in progress” and some others are artefacts to simplify the implementation of the `TiMBL` main program².

To learn more about using the API, you should study programs such as `classify.cxx`, `tse.cxx`, and the examples given in this manual, which can all be found in the `demos` directory of this distribution. As you can readily gather from these examples, the basic thing you need to do to get access to the `TimblAPI` functions is to include `timbl/TimblAPI.h` in the program, and to link against the `libtimbl` library (the command `pkg-config --cflags --libs timbl` gives the appropriate compiler and linker flags).

Important note: The described functions return a result (mostly a `bool`) to indicate success or failure. To simplify the examples, we ignore these return values. This is, of course, bad practice, to be avoided in real life programming.³

Warning: Although the `TiMBL` internals perform some sanity checking, it is quite possible to combine API functions such that some undetermined state is reached, or even a conflict arises. The effect of the `SetOptions()` function, for instance, might be quite surprising. If you have created your own program with the API it might be wise to test against well-known data to see if the results make sense.

¹<https://languagemachines.github.io/timbl>

²`Timbl.cxx` is therefore *not* a good example of how to use the API.

³as stated by commandment 6 of “The Ten Commandments for C Programmers” by Henry Spencer:

If a function be advertised to return an error code in the event of difficulties, thou shalt check for that code, yea, even though the checks triple the size of thy code and produce aches in thy typing fingers, for if thou thinkest “it cannot happen to me”, the gods shall surely punish thee for thy arrogance.

Chapter 1

Changes

1.1 From version 6.3 to 7.0

Between versions 6.3 and 7.0, TiMBL saw a long series of incremental releases. Version 6.8 brought a major refactoring of the code base that breaks both the API and the ABI; code written against earlier versions may need to be adapted, and must in any case be recompiled. Building against TiMBL now requires a C++17 compiler. The changes that affect API users are:

- TiMBL is now Unicode-based throughout: input is interpreted as UTF-8 and internally normalised to NFC. Most classification functions now exist in two variants: one taking UTF-8 encoded `std::string` arguments (as before), and one taking `icu::UnicodeString` arguments. The `Increment()` and `Decrement()` functions have `icu::UnicodeString` counterparts named `Increment_u()` and `Decrement_u()`.
- The `ValueDistribution` and `WValueDistribution` classes have been renamed to `ClassDistribution` and `WClassDistribution`. Type aliases with the old names are provided for backward compatibility. The distribution is now stored as a flat sorted vector instead of a map, which changes the iterator interface: a `dist_iterator` now dereferences to a `Vfield` directly, so you write `it->Value()` and `it->Weight()` where you used to write `it->second->Value()` and `it->second->Weight()`. See Section 3.2.
- The `StartServer()` function has been removed. All server functionality lives in the separate `TimblServer` package¹, which wraps trained TiMBL experiments in a TCP, HTTP or JSON server. See Chapter 5.
- The `ShowBestNeighbors()` function lost its second (bool) parameter; whether distributions are shown now follows from the verbosity settings.
- `CurrentWeightings()` was renamed to `GetCurrentWeights()`.
- New functions were added, reflecting new functionality of Timbl 6.4–7.0:
 - `Prune()` prunes a trained IB1-like instance base to an `IGTree`, the API equivalent of the new `--prune` command line option (see Section 4.2);
 - `SetThreads()` sets the number of threads used for testing, the equivalent of `--clones`;

¹<https://github.com/LanguageMachines/timblserver>

- `confidence()` returns the confidence of the last classification (the equivalent of the `+vcf` verbosity option, in combination with `-G`);
 - `NS_Test()` bulk-tests a file using `neighborSets`;
 - `WriteInstanceBaseLevels()` dumps only the top levels of the instance base;
 - `Expand()` and `Remove()` add or remove instances from a file to/from a trained instance base;
 - `lastHandledInstance()`, `myTargets()`, `getInputFormat()`, `isValid()`, `initExperiment()` and `BuildInfo()` give access to more of TiMBL's internal state;
 - a constructor taking a pre-parsed `TiCC::CLOptions` object was added next to the string-based constructor.
- The `Weighting` enum gained the value `SD` (standard deviation weighting for numeric features, the `-w SD` option).

1.2 From version 6.2 to 6.3

No changes to the API are made for this release. This Manual is made up to date (preserving the beta-state).

1.3 From version 6.1 to 6.2

In version 6.2, some additional functions were added to the API: `matchDepth()`, `matchedAtLeaf()`, `WriteMatrices()`, `GetMatrices()` and `ShowStatistics()`. These reflect the additional functionality of Timbl 6.2. The API is still experimental, and contains more functions than described in this manual. Using these 'undocumented' features is, as usual, unwise.

1.4 From version 5.1 to 6.1

The major change in 6.0 is the introduction of the `neighborSet` class, with some special `Classify` functions. We added `Classify` functions that deliver pointers into Timbl's internal data. This is fast, but dangerous. Also, a `WriteInstanceBaseXml()` function is added, which comes in handy when you want to know more about the instance base. Two more examples demonstrating `neighborSets` and such are added in Appendix B. From version 6.0 to 6.1, the API has not changed.

1.5 From version 5.0 to 5.1

The API is quite stable at the moment. Most TiMBL changes did not affect the API. The only real API change is in the `GetWeights()` function. (see the section on Storing and retrieving intermediate results).

Chapter 2

Quick-start

2.1 Setting up an experiment

There is just one way to start a TiMBL experiment, which is to call the `TimblAPI` constructor:

```
TimblAPI( const std::string& args, const std::string& name = "" );
```

`args` is used as a “command line” and is parsed for all kind of options which are used to create the right kind of experiment with the desired settings for metric, weighting etc. If something is wrong with the settings, *no* object is created.

The most important option is `-a` to set the kind of algorithm, e.g. `-a IB1` to invoke an IB1 experiment or `-a IGTREE` to invoke an IGTREE experiment. All command line options are described in the TiMBL Reference Guide [1].

The optional name can be useful if you have multiple experiments. In case of warnings or errors, this name is appended to the message.

For example:

```
TimblAPI *My_Experiment = new TimblAPI( "-a IGTREE +vDI+DB",  
                                         "test1" );
```

`My_Experiment` is created as an IGTREE experiment with the name “test1”, and the verbosity is set to DI+DB, meaning that the output will contain DIstance and DistriBution information.

A second constructor takes a pre-parsed `TiCC::CL_Options` object instead of an options string; this is convenient when your program passes (a subset of) its own command line on to TiMBL.

The counterpart to creation is the `~TimblAPI()` destructor, which is called when you delete an experiment:

```
delete My_Experiment;
```

2.2 Running an experiment

Assuming that we have appropriate datafiles (such as the example files `dimin.train` and `dimin.test` in the TiMBL package), we can get started right away with the functions `Learn()` and `Test()`.

2.2.1 Training

```
bool Learn( const std::string& f );
```

This function takes a file with name 'f', and gathers information such as: number of features, number and frequency of feature values and the same for class names. After that, these data are used to calculate a lot of statistical information, which will be used for testing. Finally, an `InstanceBase` is created, tuned to the current algorithm.

2.2.2 Testing

```
bool Test( const std::string& in,
           const std::string& out,
           const std::string& perc = "" );
```

Test a file given by 'in' and write results to 'out'. If 'perc' is not empty, then a percentage score is written to file 'perc'.

For example:

```
My_Experiment->Learn( "dimin.train" );
My_Experiment->Test( "dimin.test", "my_first_test" );
```

An `InstanceBase` will be created from `dimin.train`, then `dimin.test` is tested against that `InstanceBase` and output is written to `my_first_test`.

Testing can be parallelised over several threads with

```
bool SetThreads( int c );
```

which is the API equivalent of the `--clones` command line option.

2.2.3 Special cases of `Learn()` and `Test()`

There are special cases where `Learn()` behaves differently:

- When the algorithm is IB2, `Learn()` will automatically take the first n lines of `f` (set with the `-b n` option) to bootstrap itself, and then the rest of `f` for IB2-learning. After Learning IB2, you can use `Test()` as usual.

- When the algorithm is CV, `Learn()` is not defined, and all work is done in a special version of `Test()`. 'f' is assumed to give the name of a file, which, on separate lines, gives the names of the files to be cross-validated.

Also, if *featureWeights* or *probabilities* are read from user-defined datafiles, a special `CVprepare()` function must be called, to make the weighing, `weightFilename` and `probabilityFileName` known to the `Test()` function.

See Appendix B for a complete CV example (program `api_test3`).

2.3 More about settings

After an experiment is set up with the `TimblAPI` constructor, many options can be changed "on the fly" with:

```
bool SetOptions( const std::string& opts );
```

Here, 'opts' is interpreted as a list of option settings, just like in the `TimblAPI` constructor. When an error in the opts string is found, `SetOptions()` returns false. Whether any options are really set or changed in that case is undefined. Note that a few options can only be set *once* when creating the experiment, most notably the algorithm. Any attempt to change these options will result in a failure.

Note: `SetOptions()` is lazy; changes are cached until the moment they are really needed, so you can do several `SetOptions()` calls with even different values for the same option. Only the last one seen will be used for running the experiment.

To see which options are in effect, you can use the calls `ShowOptions()` and `ShowSettings()`.

```
bool ShowOptions( std::ostream& );
```

Shows all options with their possible and current values.

```
bool ShowSettings( std::ostream& );
```

Shows all options and their current values.

For example:

```
My_Experiment->SetOptions( "-w2 -m:M" );
My_Experiment->SetOptions( "-w3 -v:DB" );
My_Experiment->ShowSettings( cout )
```

See Appendix B (program `api_test1`) for the output.

2.4 Storing and retrieving intermediate results

To speed up testing, or to manipulate what is happening internally, we can store and retrieve several important parts of our experiment: The `InstanceBase`, the `FeatureWeights`, the `ProbabilityArrays` and the `ValueDistance Matrices`.

Saving is done with:

```
bool WriteInstanceBase( const std::string& f );
bool SaveWeights( const std::string& f );
bool WriteArrays( const std::string& f );
bool WriteMatrices( const std::string& f );
```

Retrieve with their counterparts:

```
bool GetInstanceBase( const std::string& f );
bool GetWeights( const std::string& f, Weighting w );
bool GetArrays( const std::string& f );
bool GetMatrices( const std::string& f );
```

All use 'f' as a filename for storing/retrieving. `GetWeights` needs information to decide *which* weighting to retrieve. Weighting is defined as the enumerated type:

```
enum Weighting { UNKNOWN_W, UD, NW, GR, IG, X2, SV, SD };
```

Some notes:

1. The InstanceBase is stored in a internal format, with or without hashing, depending on the `-H` option. The format is described in the TiMBL manual. Remember that it is a bad idea to edit this file in any way.
2. `GetWeights()` can be used to override the weights that `Learn()` calculated. `UNKNOWN_W` should not be used.
3. The Probability arrays are described in the TiMBL manual. They can be manipulated to tune the MVDm similarity metric.

If you like you may dump the Instancebase in an XML format. No Retrieve function is available for this format.

```
bool WriteInstanceBaseXml( const std::string& f );
```

Related is:

```
bool WriteInstanceBaseLevels( const std::string& f, unsigned int l );
```

which dumps only the first 'l' levels of the InstanceBase, which can be useful to inspect the global structure of a large tree.

Chapter 3

Classify functions

3.1 Classify functions: Elementary

After an experiment is trained with `Learn()`, we do not have to use `Test()` to do bulk-testing on a file. We can create our own tests with the `Classify` functions:

```
bool Classify( const std::string& Line, std::string& result );
bool Classify( const std::string& Line, std::string& result,
               double& distance );
bool Classify( const std::string& Line, std::string& result,
               std::string& Distrib, double& distance );
```

Results are stored in 'result' (the assigned class). 'distance' will get the calculated distance, and 'Distrib' the distribution at 'distance' which is used to calculate 'result'. Distrib will be a string like "{ NP 2, PP 6 }". It is up to you to parse and interpret this. (In this case: There were 8 classes assigned at 'distance', 2 NP's and 6 PP's, giving a 'result' of "PP".)

The `std::string` arguments are interpreted as UTF-8 encoded Unicode. Since version 6.8, a variant using ICU Unicode strings is also available:

```
bool Classify( const icu::UnicodeString& Line,
               icu::UnicodeString& result );
```

If you want to perform analyses on these distributions, it might be a good idea to read the next section about the other range of `Classify()` functions.

A main disadvantage compared to using `Test()` is that `Test()` is optimized. `Classify()` has to test for sanity of its input and also whether a `SetOptions()` has been performed. This slows down the process.

A good example of the use of `Classify()` is the `classify.cxx` program in the TiMBL Distribution.

Depending on the Algorithm and Verbosity setting, it may be possible to get some extra information on the details of each classification using:

```
bool ShowBestNeighbors( std::ostream& os ) const;
```

Provided that the option `+v n` or `+v k` is set and we use IB1 or IB2, output is produced similar to what we see in the TiMBL program. Bear in mind: The `+vn` option is expensive in time and memory and does not work for IGTREE, TRIBL, and TRIBL2.

Two other functions provide the results as given by the `+vmd` verbosity option:

```
size_t matchDepth() const;
bool matchedAtLeaf() const;
```

The first returns the matching Depth in the InstanceBase; the second flags whether it was a Leaf or a Non-Terminal Node.

Finally, when a normalisation is set with the `-G` option,

```
double confidence() const;
```

returns the confidence of the last classification, as given by the `+vcf` verbosity option.

3.2 Classify functions: Advanced

A faster, but more dangerous version of Classify is also available. It is faster because it returns pointers into Timbl's internal datastructures. It is dangerous because it returns pointers into Timbl's internal datastructures (using 'const' pointers, so it is fortunately difficult to really damage Timbl)

```
const TargetValue *Classify( const std::string& );
const TargetValue *Classify( const std::string&,
                             const ClassDistribution * & );
const TargetValue *Classify( const std::string&, double& );
const TargetValue *Classify( const std::string&,
                             const ClassDistribution * &,
                             double& );
```

All four also exist in a variant taking a `const icu::UnicodeString&` as their first argument.

A `ClassDistribution` (called `ValueDistribution` before version 6.8; an alias with that name is still provided) is a list-like object (but it is not a real list!) that contains `TargetValue` objects and weights. It is the result of combining all nearest neighbors and applying the desired weightings. Timbl chooses a best `TargetValue` from this `ClassDistribution` and the `Classify` functions return that as their main result.

Important: Because these functions return pointers into Timbl's internal representation, the results are only valid until the next `Classify` function is called (or the experiment is deleted).

Both the `TargetValue` and `ClassDistribution` objects have output operators defined, so you can print them. `TargetValue` also has a `name_string()` function, which returns a `std::string` so you can collect results. `ClassDistribution` has an iterator-like interface which makes it possible to walk through the `Distribution`.

An iterator on a `ClassDistribution *vd` is created like this:

```
ClassDistribution::dist_iterator it=vd->begin();
```

The iterator walks through the entries of the distribution. Every entry is a `Vfield` object; you may print it directly, or extract its `Value()` (which happens to be a `TargetValue` pointer) or extract its `Weight()`, which is a double.

Like this:

```
while ( it != vd->end() ){
    cout << *it << " has a value: ";
    cout << it->Value() << " and a weight of "
        << it->Weight() << endl;
    ++it;
}
```

Printing `*it` is the same as printing the `TargetValue` plus its `Weight`. (Before version 6.8, the iterator dereferenced to a pair, and you had to write `it->second->Value()` and `it->second->Weight()`; the `->second` part is now gone.)

In the *demos* directory you will find a complete example in `api_test6`.

Warning: it is possible to search the Timbl code for the internal representation of the `TargetValue` and `ClassDistribution` objects, but please DON'T DO THAT. The representation might change between Timbl versions.

3.3 Classify functions: `neighborSets`

A more flexible way of classifying is to use one of these functions:

```
const neighborSet *classifyNS( const icu::UnicodeString& );
bool classifyNS( const icu::UnicodeString&, neighborSet& );
bool classifyNS( const std::string&, neighborSet& );
```

The first function will classify an instance and return a pointer to a `neighborSet` object. This object may be seen as an container which holds both distances and distributions up to a certain depth, (which is *at least* the number of neighbors (-k option) that was used for the classifying task.) It is a const object, so you cannot directly manipulate its internals, but there are some functions defined to get useful information out of the `neighborSet`.

Important: The `neighborSet` *will be overwritten* on the next call to any of the classify functions. Be sure to get all the results out before that happens.

To make life easy, the other variants can be used, which fill a `neighborSet` object that you provide (the same could be achieved by a copy of the result of the first function); the `std::string` variant interprets its input as UTF-8.

Note: `NeighborSets` can be large, and copying therefore expensive, so you should only do this if you really have to.

There is also a `neighborSet`-based counterpart of `Test()`:

```
bool NS_Test( const std::string& in, const std::string& out );
```

which bulk-tests file 'in', writing per-instance `neighborSet` information to file 'out'.

3.3.1 How to get results from a neighborSet

No metric functions (such as exponential decay and the like) are performed on the neighborSet. You are free to insert your own metrics, or use Timbls built-in metrics.

```
double getDistance( size_t n ) const;
double bestDistance() const;
const ClassDistribution *getDistribution( size_t n ) const;
WClassDistribution *bestDistribution( const decayStruct * ds=0,
                                     size_t n=0 ) const ;
```

getDistance(n) will return the distance of the neighbor(s) at n. bestDistance() is simply getDistance(0).

getDistribution(n) will return the distribution of neighbor(s) at n.

bestDistribution() will return the Weighted distribution calculated using the first n elements in the container and a metric specified by the decayStruct. The default n=0, means: use the whole container. An empty decay struct means zeroDecay.

The returned WClassDistribution object is handed to you, and you are responsible for deleting it after using it (see the previous section for more details about ClassDistributions).

A decayStruct is one of:

```
class zeroDecay();
class invLinDecay();
class invDistDecay();
class expDecay( double alpha );
class expDecay( double alpha, double beta );
```

For example, to get a WClassDistribution form a neighborSet nb, using 3 neighbors and exponential decay with alpha=0.3, you can do:

```
decayStruct *dc = new expDecay(0.3);
WClassDistribution *vd = nb->bestDistribution( dc, 3 );
```

3.3.2 Useful operations on neighborSet objects

You can print neighborSet objects:

```
std::ostream& operator<<( std::ostream&, const neighborSet& );
std::ostream& operator<<( std::ostream&, const neighborSet * );
```

You may create a neighborSet yourself, and assign and delete them:

```
neighborSet();
neighborSet( const neighborSet& );
neighborSet& operator=( const neighborSet& );
~neighborSet();
```

If you create an neighborSet, you might want to reserve space for it, to avoid needless reallocations. Also it can be cleared, and you can ask the size (just like with normal containers):

```
void reserve( size_t );  
void clear();  
size_t size() const;
```

Two neighborSets can be merged:

```
void merge( const neighborSet& );
```

A neighborSet can be truncated at a certain level. This is useful after merging neighborSets. Merging sets with depth k and n will result in a set with a depth somewhere within the range $[max(k, n), k + n]$.

```
void truncate( size_t );
```

Chapter 4

Advanced Functions

4.1 Modifying the InstanceBase

The instanceBase can be modified with the functions:

```
bool Increment( const std::string& Line );
bool Decrement( const std::string& Line );
bool Increment_u( const icu::UnicodeString& Line );
bool Decrement_u( const icu::UnicodeString& Line );
```

These functions add an Instance (as described by Line) to the InstanceBase, or remove it. This can only be done for IB1-like experiments (IB1, IB2, CV and LOO), and enforces a lot of statistical recalculations. The `_u` variants take an ICU Unicode string; the `std::string` variants interpret their input as UTF-8.

More sophisticated are:

```
bool Expand( const std::string& File );
bool Remove( const std::string& File );
```

which use the contents of File to do a bulk of Increments or Decrements, and recalculate afterwards.

4.2 Pruning an InstanceBase to an IGTre

New in version 7.0 is the possibility to prune a trained (or read) IB1 or TRIBL2 instance base into an IGTre, without retraining from the original data:

```
bool Prune( bool restore_distributions = false );
```

This is the API equivalent of the `--prune` command line option. A typical use is to read a stored instance base with `GetInstanceBase()`, optionally `Expand()` it with new material, `Prune()` it, and store the result with `WriteInstanceBase()`. After pruning, the experiment behaves like an IGTre experiment. See the TiMBL Reference Guide [1] for details on the pruning process.

4.3 Getting more information out of Timbl

There are a few convenience functions to get extra information on TiMBL and its behaviour:

```
bool WriteNamesFile( const std::string& f );
```

Create a file which resembles a C4.5 namesfile.

```
Algorithm Algo()
```

Give the current algorithm as a type enum `Algorithm`. First, the declaration of the `Algorithm` type:

```
enum Algorithm { UNKNOWN_ALG, IB1, IB2, IGTREE,
                TRIBL, TRIBL2, LOO, CV };
```

This can be printed with the helper function:

```
const std::string to_string( const Algorithm )
```

```
Weighting CurrentWeighting()
```

Gives the current weighting as a type enum `Weighting`.

Declaration of `Weighting`:

```
enum Weighting { UNKNOWN_W, UD, NW, GR, IG, X2, SV, SD };
```

This can be printed with the helper function:

```
const std::string to_string( const Weighting )
```

```
Weighting GetCurrentWeights( std::vector<double>& v )
```

Returns the current weighting as a type enum `Weighting` and also a vector `v` with all the current values of this weighting. (This function was called `CurrentWeightings()` before version 6.8.)

```
std::string ExpName()
```

Returns the value of 'name' given at the construction of the experiment

```
static std::string VersionInfo( bool full = false )
```

Returns a string containing the Version number, the Revision and the Revision string of the current API implementation. If `full` is true, also information about the date and time of compilation is included.

Chapter 5

Server mode

Before version 6.4, the API contained a `StartServer()` function. All server functionality has been removed from TiMBL and lives in the separate `TimblServer` package¹, which wraps one or more trained TiMBL experiments in a network server speaking a TCP, HTTP or JSON protocol. See the `TimblServer` manual for details.

¹<https://github.com/LanguageMachines/timblserver>

Chapter 6

Annotated example programs

The example programs and the data files they expect (`dimin.train`, `dimin.test`, `small?.train` and `cross_val.test`) can be found in the `demos` directory of the TiMBL source distribution.

6.0.1 example 1, `api_test1.cxx`

```
#include "timbl/TimblAPI.h"
int main(){
    Timbl::TimblAPI My_Experiment( "-a IGTREE +vDI+DB+F", "test1" );
    My_Experiment.SetOptions( "-w3 -vDB" );
    My_Experiment.ShowSettings( std::cout );
    My_Experiment.Learn( "dimin.train" );
    My_Experiment.Test( "dimin.test", "my_first_test.out" );
    My_Experiment.SetOptions( "-mM" );
    My_Experiment.Test( "dimin.test", "my_first_test.out" );
}
```

Output (note that the settings are listed alphabetically nowadays):

```
Current Experiment Settings :
ALL_WEIGHTS           : false
BEAM_SIZE             : 0
BIN_SIZE              : 20
CLIP_FACTOR           : 10
DECAY                 : Z
DECAYPARAM_A          : 1.00000
DECAYPARAM_B          : 1.00000
DO_DIVERSIFY          : false
DO_PRUNE              : false
DO_SILLY              : false
DO_SLOPPY_LOO        : false
EXACT_MATCH           : false
EXEMPLAR_WEIGHTS     : false
FLENGTH              : 0
GLOBAL_METRIC         : 0
HANDLE_OCCURRENCES   : 0
HASHED_TREE           : true
IB2_OFFSET            : 0
IG_THRESHOLD          : 1000
IGNORE_EXEMPLAR_WEIGHTS : true
INPUTFORMAT           : Unknown
KEEP_DISTRIBUTIONS    : false
```

```

MAXBESTS           : 500
METRICS             :
MVD_LIMIT          : 1
NEIGHBORS           : 1
NO_EXEMPLAR_WEIGHTS_TEST : true
NORM_FACTOR         : 1.00000
NORMALISATION       : None
PROGRESS            : 100000
SEED                : -1
TARGET_POS          : 18446744073709551615
TREE_ORDER          : Unknown
TRIBL_OFFSET        : 0
VERBOSITY           : F+DI [Note 1]
WEIGHTING            : x2 [Note 2]

```

Examine datafile 'dimin.train' gave the following results:
Number of Features: 12
InputFormat : C4.5

```

-test1-Phase 1: Reading Datafile: dimin.train
-test1-Start:      0 @ Fri Jul  3 10:00:22 2026
-test1-Finished:   2999 @ Fri Jul  3 10:00:22 2026
-test1-Calculating Entropy      Fri Jul  3 10:00:22 2026
Lines of data      : 2999
DB Entropy         : 1.6178929
Number of Classes  : 5

```

Feats	Vals	X-square	Variance	InfoGain	GainRatio
1	3	128.41828	0.021410184	0.030971064	0.024891536
2	50	364.75812	0.030406645	0.060860038	0.027552191
3	19	212.29804	0.017697402	0.039562857	0.018676787
4	37	449.83823	0.037499019	0.052541227	0.052620750
5	3	288.87218	0.048161417	0.074523225	0.047699231
6	61	415.64113	0.034648310	0.10604433	0.024471911
7	20	501.33465	0.041791818	0.12348668	0.034953203
8	69	367.66021	0.030648567	0.097198760	0.043983864
9	2	169.36962	0.056475363	0.045752381	0.046816705
10	64	914.61906	0.076243669	0.21388759	0.042844587
11	18	2807.0418	0.23399815	0.66970458	0.18507018
12	43	7160.3682	0.59689631	1.2780762	0.32537181

```

-test1-Preparation took 0 seconds, 7 milliseconds and 343 microseconds
Feature Permutation based on Chi-Squared :
< 12, 11, 10, 7, 4, 6, 8, 2, 5, 3, 9, 1 >
-test1-Phase 2: Building multi index on Datafile: dimin.train
-test1-Start:      0 @ Fri Jul  3 10:00:22 2026
-test1-Finished:   2999 @ Fri Jul  3 10:00:22 2026
-test1-
Phase 3: Learning from Datafile: dimin.train
-test1-Start:      0 @ Fri Jul  3 10:00:22 2026
-test1-Finished:   2999 @ Fri Jul  3 10:00:22 2026

```

Size of InstanceBase = 148 Nodes, (5920 bytes), 99.61 % compression
-test1-Learning took 0 seconds, 25 milliseconds and 428 microseconds
Examine datafile 'dimin.test' gave the following results:
Number of Features: 12
InputFormat : C4.5

```

Starting to test, Testfile: dimin.test
Writing output in:      my_first_test.out
Algorithm      : IGTREE
Weighting      : Chi-square
Feature 1      : 128.418283576224439
Feature 2      : 364.758115277811896
Feature 3      : 212.298037236345095
Feature 4      : 449.838231470681876

```

```
-test1-Tested:      1 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      2 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      3 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      4 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      5 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      6 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      7 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      8 @ Fri Jul  3 10:00:22 2026
-test1-Tested:      9 @ Fri Jul  3 10:00:22 2026
-test1-Tested:     10 @ Fri Jul  3 10:00:22 2026
-test1-Tested:     100 @ Fri Jul  3 10:00:22 2026
-test1-Ready:       950 @ Fri Jul  3 10:00:22 2026
Seconds taken: 0.0719 (13214.63 p/s)
```

Warning:-test1-Metric must be Overlap for IGTREE test. [Note 3]

1. The first `SetOptions()` sets the verbosity with `+F+DI+DB`. The second `SetOptions()`, however, sets the verbosity with `-vDB`, and the resulting verbosity is therefore `F+DI`.
2. The `-w2` of the first `SetOptions()` is overruled with `-w3` from the second `SetOptions()`, resulting in a weighting of 3 or Chi-Square.
3. Due to the second `SetOptions()`, the default metric is set to `MVDM` — this is however not applicable to `IGTREE`. This raises a warning when we start to test.

=,=,=,=,=,=,=,=,+,p,e,=,T,T	6619.8512628162
=,=,=,=,=,+,k,u,=,-,bl,u,m,E,P	2396.8557978603
+,m,I,=,-,d,A,G,-,d,},t,J,J	6619.8512628162
-,t,@,=,-,l, =,-,G,@,-,T,T	6619.8512628162
-,=,I,n,-,str,y,=,-,m,E,nt,J,J	6619.8512628162
=,=,=,=,=,=,=,=,=,+,br,l,t,J,J	6619.8512628162
=,=,=,=,=,+,zw,A,=,-,m,@,r,T,T	6619.8512628162
=,=,=,=,=,-,f,u,=,+,dr,a,l,T,T	6619.8512628162
=,=,=,=,=,=,=,=,=,+,l,e,w,T,T	13780.219414719
=,=,=,=,=,+,tr,K,N,-,k,a,rt,J,J	6619.8512628162
=,=,=,=,=,+,o,=,-,p,u,=,T,T	3812.8095095379
=,=,=,=,=,=,=,=,=,+,l,A,m,E,E	3812.8095095379
=,=,=,=,=,=,=,=,=,+,l,A,p,J,J	6619.8512628162
=,=,=,=,=,=,=,=,=,+,sx,E,lm,P,P	6619.8512628162
+,l,a,=,-,d,@,=,-,k,A,st,J,J	6619.8512628162
-,s,i,=,-,f,E,r,-,st,O,k,J,J	6619.8512628162
=,=,=,=,=,=,=,=,=,+,sp,a,n,T,T	6619.8512628162
=,=,=,=,=,=,=,=,=,+,st,o,t,J,J	6619.8512628162
=,=,=,=,=,+,sp,a,r,-,b,u,k,J,J	6619.8512628162
+,h,I,N,-,k,@,l,-,bl,O,k,J,J	6619.8512628162

6.0.2 example 2, api_test2.cxx

This demonstrates IB2 learning. Our example program:

```
#include <cstdlib>
#include "timbl/TimblAPI.h"
int main(){
    Timbl::TimblAPI *My_Experiment
        = new Timbl::TimblAPI( "-a IB2 +vF+DI+DB" , "test2" );
    My_Experiment->SetOptions( "-b100" );
    My_Experiment->ShowSettings( std::cout );
    My_Experiment->Learn( "dimin.train" );
    My_Experiment->Test( "dimin.test", "my_second_test.out" );
    delete My_Experiment;
    exit(1);
}
```

We create an experiment for the IB2 algorithm, with the `-b` option set to 100, so the first 100 lines of `dimin.train` will be used to bootstrap the learning, as we can see from the output (the initial settings dump is omitted here; it is analogous to the one in example 1, with `IB2_OFFSET : 100`):

Examine datafile 'dimin.train' gave the following results:

```
Number of Features: 12
InputFormat       : C4.5
```

```
-test2-Phase 1: Reading Datafile: dimin.train
-test2-Start:      0 @ Fri Jul  3 10:00:23 2026
-test2-Finished:   2999 @ Fri Jul  3 10:00:23 2026
-test2-Calculating Entropy      Fri Jul  3 10:00:23 2026
Lines of data      : 2999 [Note 1]
DB Entropy         : 1.6178929
Number of Classes  : 5
```

Feats	Vals	InfoGain	GainRatio
1	3	0.030971064	0.024891536
2	50	0.060860038	0.027552191
3	19	0.039562857	0.018676787
4	37	0.052541227	0.052620750
5	3	0.074523225	0.047699231
6	61	0.10604433	0.024471911
7	20	0.12348668	0.034953203
8	69	0.097198760	0.043983864
9	2	0.045752381	0.046816705
10	64	0.21388759	0.042844587
11	18	0.66970458	0.18507018
12	43	1.2780762	0.32537181

```
-test2-Preparation took 0 seconds, 12 milliseconds and 393 microseconds
Feature Permutation based on GainRatio/Values :
```

```
< 9, 5, 11, 1, 12, 7, 4, 3, 10, 8, 2, 6 >
```

```
-test2-Phase 2: Learning from Datafile: dimin.train
-test2-Start:      0 @ Fri Jul  3 10:00:23 2026
-test2-Finished:   100 @ Fri Jul  3 10:00:23 2026
```

```
Size of InstanceBase = 954 Nodes, (38160 bytes), 26.62 % compression
-test2-Learning took 0 seconds, 0 milliseconds and 409 microseconds
-test2-Phase 2: Appending from Datafile: dimin.train (starting at line 101)
-test2-Start:      101 @ Fri Jul  3 10:00:23 2026
-test2-Learning:   100 @ Fri Jul  3 10:00:23 2026  added:9
-test2-Learning:   101 @ Fri Jul  3 10:00:23 2026  added:0
-test2-Learning:   102 @ Fri Jul  3 10:00:23 2026  added:0
-test2-Learning:   103 @ Fri Jul  3 10:00:23 2026  added:0
```

```

-test2-Learning:      104 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      105 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      106 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      107 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      108 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      109 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      110 @ Fri Jul 3 10:00:23 2026 added:0
-test2-Learning:      200 @ Fri Jul 3 10:00:23 2026 added:5
-test2-Learning:     1100 @ Fri Jul 3 10:00:23 2026 added:65
-test2-Finished:      2898 @ Fri Jul 3 10:00:23 2026
in total added 173 new entries

```

[Note 2]

```

Size of InstanceBase = 2232 Nodes, (89280 bytes), 32.40 % compression
DB Entropy           : 1.61789286
Number of Classes   : 5

```

```

Feats Vals InfoGain GainRatio
1          3 0.03097106 0.02489154
2          50 0.06086004 0.02755219
3          19 0.03956286 0.01867679
4          37 0.05254123 0.05262075
5           3 0.07452322 0.04769923
6          61 0.10604433 0.02447191
7          20 0.12348668 0.03495320
8          69 0.09719876 0.04398386
9           2 0.04575238 0.04681670
10         64 0.21388759 0.04284459
11         18 0.66970458 0.18507018
12         43 1.27807625 0.32537181

```

```

Examine datafile 'dimin.test' gave the following results:
Number of Features: 12
InputFormat       : C4.5

```

```

Starting to test, Testfile: dimin.test
Writing output in:      my_second_test.out
Algorithm           : IB2
Global metric       : Overlap
Deviant Feature Metrics:(none)
Weighting           : GainRatio
Feature 1  : 0.026241147173103
Feature 2  : 0.030918769841214
Feature 3  : 0.021445836516602
Feature 4  : 0.056561885447060
Feature 5  : 0.048311436541460
Feature 6  : 0.027043360641622
Feature 7  : 0.037453180788027
Feature 8  : 0.044999091421718
Feature 9  : 0.048992032381874
Feature 10 : 0.044544230779268
Feature 11 : 0.185449683494634
Feature 12 : 0.324719540921155

```

```

-test2-Tested:      1 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      2 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      3 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      4 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      5 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      6 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      7 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      8 @ Fri Jul 3 10:00:23 2026
-test2-Tested:      9 @ Fri Jul 3 10:00:23 2026
-test2-Tested:     10 @ Fri Jul 3 10:00:23 2026
-test2-Tested:     100 @ Fri Jul 3 10:00:23 2026
-test2-Ready:      950 @ Fri Jul 3 10:00:23 2026
Seconds taken: 0.1065 (8922.79 p/s)

```

overall accuracy: 0.940000 (893/950), of which 15 exact matches
There were 43 ties of which 31 (72.09%) were correctly resolved
[Note 3]

Notes:

1. IB2 is bootstrapped with 100 lines, but for the statistics all 2999 lines are used.
2. As we see here, 173 entries from the input file had a mismatch, and were therefore entered in the Instancebase.
3. We see that IB2 scores 94.00 %, compared to 96.21 % for IGTREE in our first example. For this data, IB2 is not a good algorithm. However, it saves a lot of space, and is faster than IB1. Yet, IGTREE is both faster and better. Had we used IB1, the score would have been 96.84 %.

6.0.3 example 3, api_test3.cxx

This demonstrates Cross Validation. Let's try the following program:

```
#include <cstdlib>
#include "timbl/TimblAPI.h"
using Timbl::TimblAPI;

int main(){
    TimblAPI *My_Experiment = new TimblAPI( "-t cross_validate" );
    My_Experiment->Test( "cross_val.test" );
    delete My_Experiment;
    exit(0);
}
```

This program creates an experiment, which defaults to IB1 and because of the special option “-t cross.validate” will start a CrossValidation experiment.

Learn() is not possible now. We must use a special form of Test().

“cross_val.test” is a file with the following content:

```
small_1.train
small_2.train
small_3.train
small_4.train
small_5.train
```

All these files contain an equal part of a bigger dataset, and My_Experiment will run a CrossValidation test between these files. Note that output filenames are generated and that you cannot influence that.

The output of this program is (the folds for small_2.train to small_4.train are omitted; they are analogous to the first fold):

```
Starting Cross validation test on files:
small_1.train
small_2.train
small_3.train
small_4.train
small_5.train
Examine datafile 'small_1.train' gave the following results:
Number of Features: 8
InputFormat       : C4.5
```

```
Starting to test, Testfile: small_1.train
Writing output in:          small_1.train.cv
Algorithm          : CV
Global metric      : Overlap
Deviant Feature Metrics:(none)
Weighting          : GainRatio
```

```
Tested:      1 @ Fri Jul  3 10:00:24 2026
Tested:      2 @ Fri Jul  3 10:00:24 2026
Tested:      3 @ Fri Jul  3 10:00:24 2026
Tested:      4 @ Fri Jul  3 10:00:24 2026
Tested:      5 @ Fri Jul  3 10:00:24 2026
Tested:      6 @ Fri Jul  3 10:00:24 2026
Tested:      7 @ Fri Jul  3 10:00:24 2026
Tested:      8 @ Fri Jul  3 10:00:24 2026
```

```

Tested:      9 @ Fri Jul  3 10:00:24 2026
Tested:     10 @ Fri Jul  3 10:00:24 2026
Ready:      10 @ Fri Jul  3 10:00:24 2026
Seconds taken: 0.0026 (3785.01 p/s)

overall accuracy:      0.800000  (8/10)

[... folds 2, 3 and 4: accuracies 0.800000, 0.900000, 0.800000 ...]

Examine datafile 'small_5.train' gave the following results:
Number of Features: 8
InputFormat      : C4.5

Starting to test, Testfile: small_5.train
Writing output in:      small_5.train.cv
Algorithm        : CV
Global metric    : Overlap
Deviant Feature Metrics:(none)
Weighting        : GainRatio

Tested:      1 @ Fri Jul  3 10:00:24 2026
Tested:      2 @ Fri Jul  3 10:00:24 2026
Tested:      3 @ Fri Jul  3 10:00:24 2026
Tested:      4 @ Fri Jul  3 10:00:24 2026
Tested:      5 @ Fri Jul  3 10:00:24 2026
Tested:      6 @ Fri Jul  3 10:00:24 2026
Tested:      7 @ Fri Jul  3 10:00:24 2026
Tested:      8 @ Fri Jul  3 10:00:24 2026
Ready:       8 @ Fri Jul  3 10:00:24 2026
Seconds taken: 0.0003 (29962.55 p/s)

overall accuracy:      1.000000  (8/8)

```

What has happened here?

1. TiMBL trained itself with inputfiles small_2.train through small_5.train. (in fact using the `Expand()` API call.
2. Then TiMBL tested small_1.train against the InstanceBase.
3. Next, small_2.train is removed from the database (API call `Remove()`) and small_1.train is added.
4. Then small_2.train is tested against the InstanceBase.
5. And so forth with small_3.train ...

6.0.4 example 4, api_test4.cxx

This program demonstrates adding and deleting of the InstanceBase. It also proves that weights are (re)calculated correctly each time (which also explains why this is a time-consuming thing to do). After running this program, wg.1.wgt should be equal to wg.5.wgt and wg.2.wgt equal to wg.4.wgt. Important to note is also, that while we do not use a weighting of X2 or SV here, only the “simple” weights are calculated and stored.

Further, arr.1.arr should be equal to arr.5.arr and arr.2.arr should be equal to arr.4.arr

First the program:

```
#include <cstdlib>
#include <iostream>
#include "timbl/TimblAPI.h"
using namespace Timbl;

int main(){
    TimblAPI *My_Experiment = new TimblAPI( "-a IB1 +vDI+DB +mM" ,
                                             "test4" );

    My_Experiment->ShowSettings( std::cout );
    My_Experiment->Learn( "dimin.train" );
    My_Experiment->Test( "dimin.test", "inc1.out" );
    My_Experiment->SaveWeights( "wg.1.wgt" );
    My_Experiment->WriteArrays( "arr.1.arr" );
    My_Experiment->Increment( "=", "=", "+,k,e,=-,r,@,l,T" );
    My_Experiment->Test( "dimin.test", "inc2.out" );
    My_Experiment->SaveWeights( "wg.2.wgt" );
    My_Experiment->WriteArrays( "arr.2.arr" );
    My_Experiment->Increment( "+,zw,A,rt,-,k,O,p,-,n,O,n,E" );
    My_Experiment->Test( "dimin.test", "inc3.out" );
    My_Experiment->SaveWeights( "wg.3.wgt" );
    My_Experiment->WriteArrays( "arr.3.arr" );
    My_Experiment->Decrement( "+,zw,A,rt,-,k,O,p,-,n,O,n,E" );
    My_Experiment->Test( "dimin.test", "inc4.out" );
    My_Experiment->SaveWeights( "wg.4.wgt" );
    My_Experiment->WriteArrays( "arr.4.arr" );
    My_Experiment->Decrement( "=", "=", "+,k,e,=-,r,@,l,T" );
    My_Experiment->Test( "dimin.test", "inc5.out" );
    My_Experiment->SaveWeights( "wg.5.wgt" );
    My_Experiment->WriteArrays( "arr.5.arr" );
    delete My_Experiment;
    exit(1);
}
```

This produces the following output (the settings dump and the near-identical blocks for inc2.out to inc4.out are omitted):

```
Examine datafile 'dimin.train' gave the following results:
Number of Features: 12
InputFormat       : C4.5
```

```
-test4-Phase 1: Reading Datafile: dimin.train
-test4-Start:      0 @ Fri Jul  3 11:27:09 2026
-test4-Finished:   2999 @ Fri Jul  3 11:27:09 2026
-test4-Calculating Entropy      Fri Jul  3 11:27:09 2026
-test4-Preparation took 0 seconds, 10 milliseconds and 277 microseconds
Feature Permutation based on GainRatio/Values :
< 9, 5, 11, 1, 12, 7, 4, 3, 10, 8, 2, 6 >
-test4-Phase 2: Building multi index on Datafile: dimin.train
-test4-Start:      0 @ Fri Jul  3 11:27:09 2026
-test4-Finished:   2999 @ Fri Jul  3 11:27:09 2026
```

```

-test4-
Phase 3: Learning from Datafile: dimin.train
-test4-Start:      0 @ Fri Jul  3 11:27:09 2026
-test4-Finished:   2999 @ Fri Jul  3 11:27:09 2026

Size of InstanceBase = 19231 Nodes, (769240 bytes), 49.77 % compression
-test4-Learning took 0 seconds, 33 milliseconds and 299 microseconds
Examine datafile 'dimin.test' gave the following results:
Number of Features: 12
InputFormat      : C4.5

```

```

Starting to test, Testfile: dimin.test
Writing output in:      incl.out
Algorithm      : IB1
Global metric  : Value Difference, Prestored matrix
Deviant Feature Metrics:(none)
Size of value-matrix[1] = 96 Bytes
Size of value-matrix[2] = 704 Bytes
Size of value-matrix[3] = 704 Bytes
Size of value-matrix[4] = 96 Bytes
Size of value-matrix[5] = 96 Bytes
Size of value-matrix[6] = 1496 Bytes
Size of value-matrix[7] = 1496 Bytes
Size of value-matrix[8] = 336 Bytes
Size of value-matrix[9] = 56 Bytes
Size of value-matrix[10] = 2376 Bytes
Size of value-matrix[11] = 1344 Bytes
Size of value-matrix[12] = 936 Bytes
Total Size of value-matrices 9736 Bytes

```

```

Weighting      : GainRatio

```

```

-test4-Tested:      1 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      2 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      3 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      4 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      5 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      6 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      7 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      8 @ Fri Jul  3 11:27:09 2026
-test4-Tested:      9 @ Fri Jul  3 11:27:09 2026
-test4-Tested:     10 @ Fri Jul  3 11:27:09 2026
-test4-Tested:     100 @ Fri Jul  3 11:27:09 2026
-test4-Ready:       950 @ Fri Jul  3 11:27:09 2026
Seconds taken: 0.0190 (50113.41 p/s)

```

```

overall accuracy:      0.963158 (915/950), of which 62 exact matches
There were 6 ties of which 5 (83.33%) were correctly resolved
-test4-Saving Weights in wg.1.wgt
-test4-Saving Probability Arrays in arr.1.arr

```

```

[... the blocks for inc2.out, inc3.out and inc4.out are analogous,
      with an Increment() or Decrement() preceding each of them ...]

```

```

Starting to test, Testfile: dimin.test
Writing output in:      inc5.out
Algorithm      : IB1
Global metric  : Value Difference, Prestored matrix
Deviant Feature Metrics:(none)
Size of value-matrix[1] = 96 Bytes
Size of value-matrix[2] = 704 Bytes
Size of value-matrix[3] = 704 Bytes
Size of value-matrix[4] = 96 Bytes
Size of value-matrix[5] = 96 Bytes
Size of value-matrix[6] = 1496 Bytes
Size of value-matrix[7] = 1496 Bytes

```

Size of value-matrix[8] = 336 Bytes
Size of value-matrix[9] = 56 Bytes
Size of value-matrix[10] = 2376 Bytes
Size of value-matrix[11] = 1344 Bytes
Size of value-matrix[12] = 936 Bytes
Total Size of value-matrices 9736 Bytes

Weighting : GainRatio

-test4-Tested: 1 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 2 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 3 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 4 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 5 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 6 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 7 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 8 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 9 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 10 @ Fri Jul 3 11:27:09 2026
-test4-Tested: 100 @ Fri Jul 3 11:27:09 2026
-test4-Ready: 950 @ Fri Jul 3 11:27:09 2026
Seconds taken: 0.0181 (52448.52 p/s)

overall accuracy: 0.963158 (915/950), of which 62 exact matches
There were 6 ties of which 5 (83.33%) were correctly resolved
-test4-Saving Weights in wg.5.wgt
-test4-Saving Probability Arrays in arr.5.arr

6.0.5 example 5, api_test5.cxx

This program demonstrates the use of neighborSets to classify and store results. It also demonstrates some neighborSet basics. Note the use of `icu::UnicodeString` for the instances.

```
#include <string>
#include "timbl/TimblAPI.h"

using std::endl;
using std::cout;
using std::string;
using namespace Timbl;

int main(){
    TimblAPI *My_Experiment = new TimblAPI( "-a IB1 +vDI+DB+n +mM +k4 " ,
                                             "test5" );
    My_Experiment->Learn( "dimin.train" );
    {
        icu::UnicodeString line = "=,=,=,=,+,k,e,=-,r,@,l,T";
        const neighborSet *neighbours1 = My_Experiment->classifyNS( line );
        if ( neighbours1 ){
            cout << "Classify OK on " << line << endl;
            cout << neighbours1;
        }
        else {
            cout << "Classify failed on " << line << endl;
            neighbours1 = new neighborSet();
        }
        neighborSet neighbours2;
        line = "+,zw,A,rt,-,k,O,p,-,n,O,n,E";
        if ( My_Experiment->classifyNS( line, neighbours2 ) ){
            cout << "Classify OK on " << line << endl;
            cout << neighbours2;
        }
        else {
            cout << "Classify failed on " << line << endl;
        }
        line = "+,z,O,n,-,d,A,xs,-,=,A,rm,P";
        const neighborSet *neighbours3 = My_Experiment->classifyNS( line );
        if ( neighbours3 ){
            cout << "Classify OK on " << line << endl;
            cout << neighbours3;
        }
        else {
            cout << "Classify failed on " << line << endl;
            neighbours3 = new neighborSet();
        }
        neighborSet uit2;
        {
            neighborSet uit;
            uit.setShowDistance(true);
            uit.setShowDistribution(true);
            cout << " before first merge " << endl;
            cout << uit;
            uit.merge( *neighbours1 );
            cout << " after first merge " << endl;
            cout << uit;
            uit.merge( *neighbours3 );
            cout << " after second merge " << endl;
            cout << uit;
            uit.merge( neighbours2 );
            cout << " after third merge " << endl;
            cout << uit;
            uit.truncate( 3 );
            cout << " after truncate " << endl;
            cout << uit;
        }
    }
}
```

```

        cout << " test assignment" << endl;
        uit2 = *neighbours1;
    }
    cout << "assignment result: " << endl;
    cout << uit2;
    {
        cout << " test copy construction" << endl;
        neighborSet uit(uit2);
        cout << "result: " << endl;
        cout << uit;
    }
    cout << "almost done!" << endl;
}
delete My_Experiment;
cout << "done!" << endl;
}

```

Its expected output is (without further comment):

Examine datafile 'dimin.train' gave the following results:

Number of Features: 12
InputFormat : C4.5

```

-test5-Phase 1: Reading Datafile: dimin.train
-test5-Start:      0 @ Fri Jul  3 10:00:25 2026
-test5-Finished:   2999 @ Fri Jul  3 10:00:25 2026
-test5-Calculating Entropy      Fri Jul  3 10:00:25 2026
-test5-Preparation took 0 seconds, 12 milliseconds and 787 microseconds
Feature Permutation based on GainRatio/Values :
< 9, 5, 11, 1, 12, 7, 4, 3, 10, 8, 2, 6 >
-test5-Phase 2: Building multi index on Datafile: dimin.train
-test5-Start:      0 @ Fri Jul  3 10:00:25 2026
-test5-Finished:   2999 @ Fri Jul  3 10:00:25 2026
-test5-
Phase 3: Learning from Datafile: dimin.train
-test5-Start:      0 @ Fri Jul  3 10:00:25 2026
-test5-Finished:   2999 @ Fri Jul  3 10:00:25 2026

```

Size of InstanceBase = 19231 Nodes, (769240 bytes), 49.77 % compression

-test5-Learning took 0 seconds, 29 milliseconds and 875 microseconds

Classify OK on =,=,=,=,+,k,e,=-,r,@,l,T

k=1 { T 1.00000 } 0.000000000000000

k=2 { T 1.00000 } 0.00318629024734

k=3 { T 1.00000 } 0.00341823151183

k=4 { T 1.00000 } 0.00374337728446

Classify OK on +,zw,A,rt,-,k,O,p,-,n,O,n,E

k=1 { E 1.00000 } 0.000000000000000

k=2 { E 1.00000 } 0.05666788032719

k=3 { E 1.00000 } 0.06255263661774

k=4 { E 1.00000 } 0.06442386036189

Classify OK on +,z,O,n,-,d,A,xs,-,=,A,rm,P

k=1 { P 1.00000 } 0.05972983625517

k=2 { P 1.00000 } 0.08774076913265

k=3 { P 1.00000 } 0.08844278891972

k=4 { P 1.00000 } 0.09705864995143

before first merge

after first merge

k=1 { P 1.00000 } 0.05972983625517

k=2 { P 1.00000 } 0.08774076913265

k=3 { P 1.00000 } 0.08844278891972

k=4 { P 1.00000 } 0.09705864995143

after second merge

k=1 { P 2.00000 } 0.05972983625517

k=2 { P 2.00000 } 0.08774076913265

k=3 { P 2.00000 } 0.08844278891972

k=4 { P 2.00000 } 0.09705864995143

```

after third merge
# k=1 { E 1.00000 } 0.000000000000000
# k=2 { E 1.00000 } 0.05666788032719
# k=3 { P 2.00000 } 0.05972983625517
# k=4 { E 1.00000 } 0.06255263661774
# k=5 { E 1.00000 } 0.06442386036189
# k=6 { P 2.00000 } 0.08774076913265
# k=7 { P 2.00000 } 0.08844278891972
# k=8 { P 2.00000 } 0.09705864995143
after truncate
# k=1 { E 1.00000 } 0.000000000000000
# k=2 { E 1.00000 } 0.05666788032719
# k=3 { P 2.00000 } 0.05972983625517
test assignment
assignment result:
# k=1 { P 1.00000 } 0.05972983625517
# k=2 { P 1.00000 } 0.08774076913265
# k=3 { P 1.00000 } 0.08844278891972
# k=4 { P 1.00000 } 0.09705864995143
test copy construction
result:
# k=1 { P 1.00000 } 0.05972983625517
# k=2 { P 1.00000 } 0.08774076913265
# k=3 { P 1.00000 } 0.08844278891972
# k=4 { P 1.00000 } 0.09705864995143
almost done!
done!

```

6.0.6 example 6, api_test6.cxx

This program demonstrates the use of ClassDistributions, TargetValues and neighborSets for classification. Note the new iterator interface (`*it, it->Value()`, `it->Weight()`) and the use of a normalisation (`-G 0`), which makes the distribution weights probabilities.

```
#include <iostream>
#include "timbl/TimblAPI.h"

using std::cout;
using std::endl;
using namespace Timbl;

int main(){
    TimblAPI My_Experiment( "-a IB1 +vDI+DB -G 0 -k3", "test6" );
    My_Experiment.Learn( "dimin.train" );
    const ClassDistribution *vd;
    const TargetValue *tv
        = My_Experiment.Classify( std::string("-",O,m,+,h,K,=-,n,I,N,K"), vd );
    cout << "resulting target: " << tv << endl;
    cout << "resulting Distribution: " << vd << endl;
    ClassDistribution::dist_iterator it=vd->begin();
    while ( it != vd->end() ){
        cout << *it << " OR ";
        cout << it->Value() << " " << it->Weight() << endl;
        ++it;
    }

    cout << "the same with neighborSets" << endl;
    const neighborSet *nb = My_Experiment.classifyNS( "-",O,m,+,h,K,=-,n,I,N,K" );
    WClassDistribution *vd2 = nb->bestDistribution();
    vd2->Normalize();
    cout << "default answer " << vd2 << endl;
    decayStruct *dc = new expDecay(0.3);
    delete vd2;
    vd2 = nb->bestDistribution( dc );
    delete dc;
    cout << "with exponential decay, alpha = 0.3 " << vd2 << endl;
    delete vd2;
}
```

This is the output produced:

```
Examine datafile 'dimin.train' gave the following results:
Number of Features: 12
InputFormat       : C4.5

-test6-Phase 1: Reading Datafile: dimin.train
-test6-Start:      0 @ Fri Jul  3 11:27:14 2026
-test6-Finished:   2999 @ Fri Jul  3 11:27:14 2026
-test6-Calculating Entropy      Fri Jul  3 11:27:14 2026
-test6-Preparation took 0 seconds, 11 milliseconds and 245 microseconds
Feature Permutation based on GainRatio/Values :
< 9, 5, 11, 1, 12, 7, 4, 3, 10, 8, 2, 6 >
-test6-Phase 2: Building multi index on Datafile: dimin.train
-test6-Start:      0 @ Fri Jul  3 11:27:14 2026
-test6-Finished:   2999 @ Fri Jul  3 11:27:14 2026
-test6-
Phase 3: Learning from Datafile: dimin.train
-test6-Start:      0 @ Fri Jul  3 11:27:14 2026
-test6-Finished:   2999 @ Fri Jul  3 11:27:14 2026

Size of InstanceBase = 19231 Nodes, (769240 bytes), 49.77 % compression
```

```
-test6-Learning took 0 seconds, 29 milliseconds and 168 microseconds  
resulting target: K  
resulting Distribution: { E 0.125000, K 0.875000 }  
E 0.125000 OR E 0.125000  
K 0.875000 OR K 0.875000  
the same with neighborSets  
default answer { E 0.125000, K 0.875000 }  
with exponential decay, alpha = 0.3 { E 0.971556, K 6.69810 }
```

Bibliography

- [1] Walter Daelemans, Jakub Zavrel, Ko van der Sloot, and Antal van den Bosch (2026). *TiMBL: Tilburg Memory-Based Learner, version 7.0, Reference Guide*. Available from https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl_7.0_Manual.pdf.